

# Ferramentas modernas em R: introdução ao tidyverse e tidymodels

Mineração de Dados

---

Helton Graziadei

# Por que precisamos do `tidymodels`?

Vamos ajustar um KNN simples ao dataset `Default` (ISLR2) de duas formas:

1. **R base**: pré-processamento manual, passo a passo.
2. **tidymodels**: receita + workflow.

Ao comparar os dois caminhos, vamos observar:

- O código manual é longo e frágil: fácil esquecer um passo ou aplicá-lo errado.
- O risco de *data leakage* é real e silencioso.
- O `tidymodels` encapsula boas práticas em poucas linhas.

## Caminho 1: R base — preparação manual (parte 1)

```
library(ISLR2)
library(class) # knn do R base

dados <- Default

# 1. Converter 'student' em numerico manualmente
dados$student_num <- ifelse(dados$student == "Yes", 1, 0)

# 2. Dividir treino/teste (sem estratificacao!)
set.seed(42)
idx <- sample(1:nrow(dados), size = 0.75 * nrow(dados))
treino <- dados[idx, ]
teste <- dados[-idx, ]
```

### Problema 1

A divisão com `sample()` não estratifica pela resposta. Se `default = "Yes"` é raro ( $\approx 3\%$ ), o teste pode ficar com proporção muito diferente.

## Caminho 1: R base — preparação manual (parte 2)

```
# 3. Padronizar variaveis numericas
#   ERRO COMUM: usar media/desvio de TODOS os dados!
media_bal <- mean(treino$balance)
dp_bal    <- sd(treino$balance)
media_inc <- mean(treino$income)
dp_inc    <- sd(treino$income)

treino$bal_z <- (treino$balance - media_bal) / dp_bal
treino$inc_z <- (treino$income - media_inc) / dp_inc
teste$bal_z  <- (teste$balance - media_bal) / dp_bal
teste$inc_z  <- (teste$income - media_inc) / dp_inc

# 4. Montar matrizes de preditores
X_treino <- as.matrix(treino[, c("bal_z", "inc_z", "student_num")])
X_teste  <- as.matrix(teste[, c("bal_z", "inc_z", "student_num")])
y_treino <- treino$default
```

### Problema 2

São **10 linhas** só para padronizar 2 variáveis. Com 20 preditores, esse bloco explode — e cada linha é uma chance de erro (trocar treino por dados, esquecer uma variável...).

## Caminho 1: R base — ajuste e avaliação

```
# 5. Ajustar KNN
```

```
pred_knn <- knn(train = X_treino, test = X_teste,  
                cl = y_treino, k = 5)
```

```
# 6. Avaliar
```

```
tab <- table(Predito = pred_knn, Real = teste$default)
```

```
tab
```

```
acuracia <- sum(diag(tab)) / sum(tab)
```

```
acuracia
```

## Caminho 1: R base — ajuste e avaliação

```
# 5. Ajustar KNN
pred_knn <- knn(train = X_treino, test = X_teste,
               cl = y_treino, k = 5)

# 6. Avaliar
tab <- table(Predito = pred_knn, Real = teste$default)
tab
acuracia <- sum(diag(tab)) / sum(tab)
acuracia
```

### Caminho manual via R base

- ~25 linhas de código.
- Padronização, codificação e divisão feitas “na mão”.
- Se quisermos tunar  $K$  ou trocar o modelo, precisamos reescrever boa parte do código.
- Se esquecermos de usar as médias do treino no teste  $\Rightarrow$  *data leakage* silencioso.

## Caminho 2: tidymodels — tudo em ~15 linhas

```
library(tidymodels)
dados <- as_tibble(ISLR2::Default)

# Divisao estratificada
set.seed(42)
split <- initial_split(dados, prop = 0.75, strata = default)
treino <- training(split)
teste <- testing(split)

# Receita: pre-processamento declarativo
receita <- recipe(default ~., data = treino) |>
  step_dummy(all_nominal_predictors()) |>
  step_zv(all_predictors()) |>
  step_normalize(all_numeric_predictors())

# Modelo + workflow + ajuste
wf_fit <- workflow() |>
  add_recipe(receita) |>
  add_model(nearest_neighbor(neighbors = 5) |>
    set_engine("kkn") |> set_mode("classification")) |>
  fit(data = treino)
```

## Caminho 2: avaliação

```
# Predicao + metricas em poucas linhas
resultados <-teste |>
  bind_cols(predict(wf_fit, new_data = teste))

resultados |>accuracy(truth = default, estimate = .pred_class)
resultados |>conf_mat(truth = default, estimate = .pred_class)
```



## Caminho 2: avaliação

```
# Predicao + metricas em poucas linhas
resultados <-teste |>
  bind_cols(predict(wf_fit, new_data = teste))

resultados |>accuracy(truth = default, estimate = .pred_class)
resultados |>conf_mat(truth = default, estimate = .pred_class)
```

### Vantagens

- Divisão estratificada com uma linha.
- Pré-processamento declarativo: sem risco de *leakage*.
- Para trocar o modelo, basta mudar uma linha (ex: `logistic_reg()`).
- Para tunar  $K$ : substituir `neighbors = 5` por `tune()` e adicionar `tune_grid()`.

|                          | R base               | tidymodels                          |
|--------------------------|----------------------|-------------------------------------|
| Divisão estratificada    | manual / difícil     | <code>initial_split(strata=)</code> |
| Pré-processamento seguro | responsabilidade sua | automático via <code>recipe</code>  |
| Risco de <i>leakage</i>  | alto (silencioso)    | eliminado por design                |
| Trocar modelo            | reescrever tudo      | trocar 1 linha                      |
| Tunar hiperparâmetros    | loop manual          | <code>tune_grid()</code>            |
| Linhas de código (KNN)   | ~25                  | ~15                                 |

Vamos aprender cada peça do `tidymodels` em detalhe.

# Importando dados com readr

O pacote readr (parte do tidyverse) oferece funções rápidas e robustas para leitura de dados tabulares.

```
library(tidyverse)

# CSV com separador virgula (padrao internacional)
dados <- read_csv("caminho/para/arquivo.csv")

# CSV com separador ponto-e-virgula (padrao brasileiro)
dados <- read_csv2("caminho/para/arquivo.csv")
```

## Por que read\_csv e não read.csv?

- Retorna um tibble em vez de data.frame.
- Mais rápido para arquivos grandes.
- Não converte strings em factor automaticamente.
- Mostra um resumo das colunas importadas.

## Exemplo: lendo o dataset Default (ISLR)

```
library(ISLR2)
dados <- as_tibble(Default)
dados
```

Saída típica:

```
# A tibble: 10,000 x 3
  default balance  income
  <fct>   <dbl>   <dbl>
1 No      730.   44362.
2 No      817.   12106.
3 No     1074.   31767.
4 No      529.   35704.
...
```

Observe: o tibble já nos informa dimensões, tipos de cada coluna e exibe apenas as primeiras linhas. Mais informativo que um `data.frame` padrão.

# O que é um Tibble?

Um tibble é a versão moderna do `data.frame` no R:

- Mostra apenas as primeiras 10 linhas e as colunas que cabem na tela.

# O que é um Tibble?

Um tibble é a versão moderna do `data.frame` no R:

- Mostra apenas as primeiras 10 linhas e as colunas que cabem na tela.
- Não faz conversão automática de tipos (`stringsAsFactors = FALSE`).

# O que é um Tibble?

Um tibble é a versão moderna do `data.frame` no R:

- Mostra apenas as primeiras 10 linhas e as colunas que cabem na tela.
- Não faz conversão automática de tipos (`stringsAsFactors = FALSE`).
- `df[, 1]` sempre retorna um tibble, nunca um vetor.

# O que é um Tibble?

Um tibble é a versão moderna do `data.frame` no R:

- Mostra apenas as primeiras 10 linhas e as colunas que cabem na tela.
- Não faz conversão automática de tipos (`stringsAsFactors = FALSE`).
- `df[, 1]` sempre retorna um tibble, nunca um vetor.
- Qualquer função que aceita `data.frame` aceita tibble.



# O que é um Tibble?

Um tibble é a versão moderna do `data.frame` no R:

- Mostra apenas as primeiras 10 linhas e as colunas que cabem na tela.
- Não faz conversão automática de tipos (`stringsAsFactors = FALSE`).
- `df[, 1]` sempre retorna um tibble, nunca um vetor.
- Qualquer função que aceita `data.frame` aceita tibble.

```
# Criando um tibble manualmente
tibble(
  nome = c("Ana", "Bruno", "Carla"),
  nota = c(8.5, 7.0, 9.2),
  turma = c("A", "B", "A")
)
```

## glimpse() — visão rápida

```
glimpse(dados)
```

```
Rows: 10,000  
Columns: 4  
$ default <fct> No, No, ...  
$ student <fct> No, Yes, ...  
$ balance <dbl> 730, 817, ...  
$ income <dbl> 44362, 12106,...
```

Mostra cada variável como uma linha: tipo e primeiros valores.

## skim() — resumo completo

```
library(skimr)  
skim(dados)
```

Retorna uma tabela rica com:

- Contagens de NA
- Média, desvio-padrão, quartis
- Histogramas em texto
- Resumos por tipo de variável

# Os verbos principais do dplyr



`select()`

Escolhe colunas



`filter()`

Filtra linhas por condição



`mutate()`

Cria/transforma colunas



`arrange()`

Ordena linhas



`group_by()`

Agrupar por variável



`summarise()`

Resume em estatísticas



`count()`

Conta observações



`rename()`

Renomeia colunas

# Verbos na prática

```
# Selecionar colunas e filtrar linhas
dados |>
  select(default, balance, income) |>
  filter(balance > 1500)

# Criar nova variavel
dados |>
  mutate(bal_log = log(balance + 1))

# Resumo por grupo
dados |>
  group_by(default) |>
  summarise(
    media_bal = mean(balance),
    media_inc = mean(income),
    n = n()
  )
```

# O operador `|>` (pipe)

O pipe passa o resultado da expressão à esquerda como primeiro argumento da função à direita.

## Sem pipe (aninhado):

```
arrange(  
  summarise(  
    group_by(dados, default),  
    m = mean(balance)  
  ),  
  desc(m)  
)
```

Leitura de dentro para fora. Difícil de seguir!

## Com pipe (encadeado):

```
dados |>  
  group_by(default) |>  
  summarise(m = mean(balance)) |>  
  arrange(desc(m))
```

Leitura de cima para baixo. Cada linha é um passo!

## Dois pipes no R

`|>` é o pipe nativo (R  $\geq$  4.1). O `%>%` do `magrittr` ainda funciona, mas o nativo é preferido em código novo.

## O que são *Tidy Data* (Dados Arrumados)?

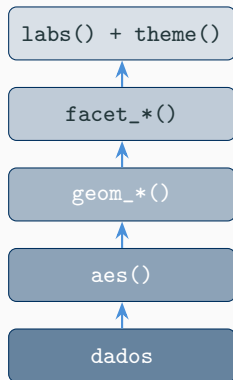
Antes de treinar modelos, a estrutura dos dados deve ser padronizada. O conceito de *Tidy Data* (Wickham, 2014) baseia-se em três princípios fundamentais:

1. Cada variável deve ter sua própria coluna.
2. Cada observação deve ter sua própria linha.
3. Cada valor deve ter sua própria célula.

É muito comum recebermos dados onde os *valores* de uma variável estão nos nomes das colunas (ex: colunas separadas para os anos “2024” e “2025”).

Algoritmos de aprendizado estatístico e pacotes de visualização assumem que os dados de entrada estão no formato *tidy*. Manter esse padrão elimina a necessidade de reescrever código de manipulação para cada nova base de dados.

# A gramática dos gráficos



Rótulos e aparência

Painéis por variável categórica

Geometria: pontos, barras, linhas. . .

Mapeamento: x, y, cor, forma

Data frame / tibble

# Exemplos essenciais

```
# Disperso com cor por grupo
ggplot(dados, aes(x = balance, y = income, color = default)) +
  geom_point(alpha = 0.4) +
  labs(title = "Saldo vs. Renda", x = "Saldo", y = "Renda") +
  theme_minimal()

# Boxplot comparativo
ggplot(dados, aes(x = default, y = balance, fill = default)) +
  geom_boxplot() +
  labs(title = "Distribuio do Saldo por Inadimplncia") +
  theme_minimal()

# Histograma com facetas
ggplot(dados, aes(x = balance)) +
  geom_histogram(bins = 30, fill = "steelblue", color = "white") +
  facet_wrap(~ default, scales = "free_y") +
  theme_minimal()
```



| Objetivo                | Geometria                     | Quando usar                        |
|-------------------------|-------------------------------|------------------------------------|
| Relação entre numéricas | <code>geom_point()</code>     | Scatter plots, detecção de padrões |
| Tendência / ajuste      | <code>geom_smooth()</code>    | Curvas suavizadas ou regressão     |
| Distribuição contínua   | <code>geom_histogram()</code> | Uma variável numérica              |
| Comparação de grupos    | <code>geom_boxplot()</code>   | Numérica $\times$ categórica       |
| Distribuição suavizada  | <code>geom_density()</code>   | Comparar densidades                |
| Contagem de categorias  | <code>geom_bar()</code>       | Frequência de classes              |
| Séries temporais        | <code>geom_line()</code>      | Dados ordenados no tempo           |

# O que é pré-processamento?

Antes de ajustar um modelo, os dados geralmente precisam de transformações:

- Normalização / padronização de variáveis numéricas.

# O que é pré-processamento?

Antes de ajustar um modelo, os dados geralmente precisam de transformações:

- Normalização / padronização de variáveis numéricas.
- Codificação de variáveis categóricas.

# O que é pré-processamento?

Antes de ajustar um modelo, os dados geralmente precisam de transformações:

- Normalização / padronização de variáveis numéricas.
- Codificação de variáveis categóricas.
- Imputação de dados faltantes.

# O que é pré-processamento?

Antes de ajustar um modelo, os dados geralmente precisam de transformações:

- Normalização / padronização de variáveis numéricas.
- Codificação de variáveis categóricas.
- Imputação de dados faltantes.
- Remoção de variáveis com variância zero.

# O que é pré-processamento?

Antes de ajustar um modelo, os dados geralmente precisam de transformações:

- Normalização / padronização de variáveis numéricas.
- Codificação de variáveis categóricas.
- Imputação de dados faltantes.
- Remoção de variáveis com variância zero.
- Seleção / redução de dimensionalidade.

# O que é pré-processamento?

Antes de ajustar um modelo, os dados geralmente precisam de transformações:

- Normalização / padronização de variáveis numéricas.
- Codificação de variáveis categóricas.
- Imputação de dados faltantes.
- Remoção de variáveis com variância zero.
- Seleção / redução de dimensionalidade.

## Pergunta

Quando devemos fazer essas transformações. Antes ou depois de dividir os dados em treino e teste?

# Vazamento de Informação (*data leakage*)

✗ **ERRADO**

Padronizar **todos** os dados



Dividir treino/teste



Treinar modelo



**Métricas otimistas!**

✓ **CORRETO**

Dividir treino/teste



Padronizar usando **só o treino**



Aplicar mesmos parâmetros ao teste



**Avaliação honesta!**

*Data leakage* ocorre quando informação do conjunto de teste “vaza” para o treinamento. Isso infla artificialmente as métricas de desempenho.

## Exemplo

Se calcularmos a média e o desvio-padrão de uma variável usando todos os dados, a padronização do treino carrega informação do teste.



# Uma solução integrada: tidymodels



O tidymodels é uma coleção de pacotes que fornece uma interface consistente e segura para todo o fluxo de modelagem, da divisão dos dados à avaliação final.

# Dividindo com rsample

```
library(tidymodels)

set.seed(42)

# Divisao estratificada: mantem proporcao de 'default'
split <- initial_split(dados, prop = 0.75, strata = default)

treino <- training(split)
teste <- testing(split)

nrow(treino)
nrow(teste)
```

## Por que estratificar?

Se a variável resposta é desbalanceada (ex: apenas 3% de inadimplentes), uma divisão aleatória simples pode gerar amostras com proporções muito diferentes. A estratificação garante que treino e teste reflitam a distribuição original.

# Definindo uma receita (recipe)

Uma receita é uma sequência de passos de pré-processamento declarada *antes* de ver os dados:

```
receita <- recipe(default ~., data = treino) |>  
  step_dummy(all_nominal_predictors()) |> # dummies  
  step_zv(all_predictors()) |>           # remove var. zero  
  step_normalize(all_numeric_predictors()) # padroniza
```

- A fórmula default `~ .` define a resposta e os preditores.

# Definindo uma receita (recipe)

Uma receita é uma sequência de passos de pré-processamento declarada *antes* de ver os dados:

```
receita <- recipe(default ~., data = treino) |>  
  step_dummy(all_nominal_predictors()) |> # dummies  
  step_zv(all_predictors()) |>           # remove var. zero  
  step_normalize(all_numeric_predictors()) # padroniza
```

- A fórmula default  $\sim .$  define a resposta e os preditores.
- Cada step\_\*() descreve uma transformação.

# Definindo uma receita (recipe)

Uma receita é uma sequência de passos de pré-processamento declarada *antes* de ver os dados:

```
receita <- recipe(default ~., data = treino) |>  
  step_dummy(all_nominal_predictors()) |> # dummies  
  step_zv(all_predictors()) |>           # remove var. zero  
  step_normalize(all_numeric_predictors()) # padroniza
```

- A fórmula default `~ .` define a resposta e os preditores.
- Cada `step_*()` descreve uma transformação.
- Nenhum cálculo é feito ainda. A receita é apenas um **plano**.

# Definindo uma receita (recipe)

Uma receita é uma sequência de passos de pré-processamento declarada *antes* de ver os dados:

```
receita <- recipe(default ~., data = treino) |>  
  step_dummy(all_nominal_predictors()) |> # dummies  
  step_zv(all_predictors()) |>          # remove var. zero  
  step_normalize(all_numeric_predictors()) # padroniza
```

- A fórmula default ~ . define a resposta e os preditores.
- Cada step\_\*() descreve uma transformação.
- Nenhum cálculo é feito ainda. A receita é apenas um **plano**.

## Steps úteis

step\_impute\_median(), step\_impute\_mode(), step\_log(), step\_corr(), step\_pca(), step\_interact(), entre muitos outros.

# prep() e bake(): executando a receita

## Três etapas:

1. `recipe()`: define os passos de pré-processamento (é apenas um **plano**).
2. `prep()`: estima os parâmetros (média, desvio, etc.) usando **só o treino**.
3. `bake()`: aplica a receita preparada a dados novos.

```
# Estimar parâmetros (mdia, desvio, etc.) NO TREINO
receita_prep <- prep(receita, training = treino)

# Aplicar ao treino (equivalente a juice())
treino_proc <- bake(receita_prep, new_data = NULL)

# Aplicar ao teste (usa parâmetros do treino!)
teste_proc <- bake(receita_prep, new_data = teste)
```

## Isso evita data leakage!

O `prep()` aprende estatísticas **apenas** com o treino. O `bake()` aplica essas mesmas estatísticas ao teste, sem recalcular.

## prep() e bake(): executando a receita

```
# Estimar parametros (media, desvio, etc.) NO TREINO
receita_prep <- prep(receita, training = treino)
# Aplicar ao treino (equivalente a juice())
treino_proc <- bake(receita_prep, new_data = NULL)
# Aplicar ao teste (usa parametros do treino!)
teste_proc <- bake(receita_prep, new_data = teste)
```

### Isso evita data leakage!

O `prep()` aprende estatísticas apenas com o treino. O `bake()` aplica essas mesmas estatísticas ao teste, sem recalcular.



# Por que parsnip?

No R base, cada pacote tem sua própria sintaxe para ajustar modelos:

| Modelo        | Pacote  | Função      |
|---------------|---------|-------------|
| KNN           | kknn    | kknn()      |
| Random Forest | ranger  | ranger()    |
| Boosting      | xgboost | xgb.train() |
| SVM           | kernlab | ksvm()      |

# Por que parsnip?

No R base, cada pacote tem sua própria sintaxe para ajustar modelos:

| Modelo        | Pacote  | Função      |
|---------------|---------|-------------|
| KNN           | kknn    | kknn()      |
| Random Forest | ranger  | ranger()    |
| Boosting      | xgboost | xgb.train() |
| SVM           | kernlab | ksvm()      |

O parsnip oferece uma interface unificada:

```
modelo <-nearest_neighbor(neighbors = 5) |>  
  set_engine("kknn") |>  
  set_mode("classification")
```

- `nearest_neighbor()`: especifica o *tipo* de modelo.

# Por que parsnip?

No R base, cada pacote tem sua própria sintaxe para ajustar modelos:

| Modelo        | Pacote  | Função      |
|---------------|---------|-------------|
| KNN           | kkn     | kkn()       |
| Random Forest | ranger  | ranger()    |
| Boosting      | xgboost | xgb.train() |
| SVM           | kernlab | ksvm()      |

O parsnip oferece uma interface unificada:

```
modelo <-nearest_neighbor(neighbors = 5) |>  
  set_engine("kkn") |>  
  set_mode("classification")
```

- `nearest_neighbor()`: especifica o *tipo* de modelo.
- `set_engine()`: escolhe a implementação (pacote por trás).

# Por que parsnip?

No R base, cada pacote tem sua própria sintaxe para ajustar modelos:

| Modelo        | Pacote  | Função      |
|---------------|---------|-------------|
| KNN           | kkn     | kkn()       |
| Random Forest | ranger  | ranger()    |
| Boosting      | xgboost | xgb.train() |
| SVM           | kernlab | ksvm()      |

O parsnip oferece uma interface unificada:

```
modelo <-nearest_neighbor(neighbors = 5) |>  
  set_engine("kkn") |>  
  set_mode("classification")
```

- `nearest_neighbor()`: especifica o *tipo* de modelo.
- `set_engine()`: escolhe a implementação (pacote por trás).
- `set_mode()`: classificação ou regressão.

# KNN com parsnip + workflow

Um workflow une receita + modelo em um único objeto:

```
# Definir modelo
knn_spec <- nearest_neighbor(neighbors = 5) |>
  set_engine("kkn") |>
  set_mode("classification")

# Criar workflow
wf <- workflow() |>
  add_recipe(receita) |>
  add_model(knn_spec)

# Ajustar no treino
wf_fit <- fit(wf, data = treino)

# Predizer no teste
predicoes <- predict(wf_fit, new_data = teste)
```

O workflow cuida de tudo: aplica prep/bake automaticamente antes de ajustar ou predizer.

# Coletando métricas no teste

```
resultados <-teste |>
  bind_cols(predict(wf_fit, new_data = teste)) |>
  bind_cols(predict(wf_fit, new_data = teste, type = "prob"))

# Acurcia
resultados |>accuracy(truth = default, estimate = .pred_class)

# Matriz de confuso
resultados |>conf_mat(truth = default, estimate = .pred_class)

# AUC-ROC
resultados |>roc_auc(truth = default, .pred_Yes)

# Vrias mtricas de uma vez
metricas <-metric_set(accuracy, roc_auc)
resultados |>metricas(truth = default, .pred_class, .pred_Yes)
```

# Tunando hiperparâmetros com validação cruzada

```
# Modelo com hiperparametro a tunar
knn_tune <-nearest_neighbor(neighbors = tune()) |>
  set_engine("kknn") |>
  set_mode("classification")

# Workflow atualizado
wf_tune <-workflow() |>
  add_recipe(receita) |>
  add_model(knn_tune)

# Reamostragem: 10-fold CV estratificada
folds <-vfold_cv(treino, v = 10, strata = default)

# Grid search
resultados_tune <-tune_grid(
  wf_tune,
  resamples = folds,
  grid = tibble(neighbors = c(1, 3, 5, 7, 9, 11, 15, 21, 31))
)
```

# Selecionando o melhor modelo

```
# Visualizar resultados do tuning
autoplot(resultados_tune)

# Selecionar melhor K pela AUC-ROC
melhor <-select_best(resultados_tune, metric = "roc_auc")
melhor
```

```
# Finalizar workflow com o melhor hiperparametro
wf_final <-finalize_workflow(wf_tune, melhor)

# Ajuste final: treina no treino, avalia no teste
ajuste_final <-last_fit(wf_final, split)

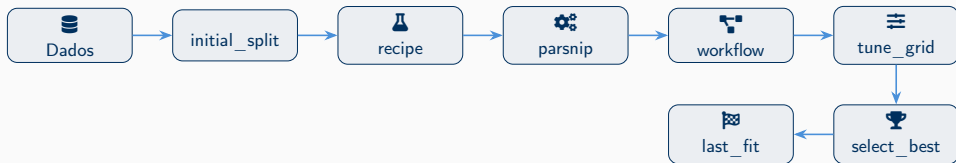
# Mtricas finais (avaliadas no teste)
collect_metrics(ajuste_final)
```

## last\_fit()

A função `last_fit()` faz tudo de uma vez: treina no conjunto de treino e avalia no conjunto de teste, reportando métricas honestas.



# O fluxo completo



1. Ler e explorar os dados (`read_csv`, `glimpse`, `skim`).
2. Dividir em treino/teste com `initial_split()`.
3. Definir receita de pré-processamento (`recipe`).
4. Especificar modelo com `parsnip`.
5. Unir tudo em um `workflow`.
6. Tunar via validação cruzada (`tune_grid`).
7. Selecionar o melhor e avaliar com `last_fit()`.

- Wickham, H. & Grolemund, G. *R for Data Science* (2ª ed.), <https://r4ds.hadley.nz>
- Kuhn, M. & Silge, J. *Tidy Modeling with R*, <https://www.tmwr.org>
- James, G. et al. *An Introduction to Statistical Learning* (ISLR2), Cap. 2–4.
- Documentação: <https://www.tidymodels.org>